

TEMA 2. Encapsulación

1. En Programación Orientada a Objetos (POO), ¿Qué buscan la **encapsulación** y la **ocultación de información**? Enumera brevemente algunas ventajas de la **ocultación de información**.

En POO tanto la encapsulación como la ocultación de información buscan proteger y organizar mejor los datos y el comportamiento de los objetos, pero cada concepto pone el foco en algo distinto:

ENCAPSULACIÓN

La encapsulación consiste en agrupar los datos (atributos) y los métodos que operan sobre ellos dentro de una clase, y controlar el acceso a esos datos mediante modificadores (como `private`, `protected` o `public`)

OCULTACIÓN DE INFORMACIÓN

La ocultación de información busca esconder los detalles internos de implementación de un objeto, mostrando solo lo necesario a través de una interfaz pública.

Alguna de sus ventajas son:

- Reduce el acoplamiento
- Mejora la mantenibilidad
- Aumenta la seguridad
- Facilita la reutilización
- Mejora la comprensión del código

****CLASE

La encapsulación tiene que ver con "Protección":

- Evito estados no válidos de mis objetos
- Evito dependencias desde fuera que no quiero

2 Partes-> He juntado estado y comportamiento en un artefacto (la clase), y ahora puedo ocultar ciertas partes del exterior

2. ¿Qué se entiende por la **interfaz pública** de un objeto o clase en POO? Describe brevemente cómo se relaciona con la ocultación de información.

La interfaz pública de un objeto o clase es el conjunto de métodos accesibles desde fuera de la clase, es decir, aquello que otros objetos pueden usar para interactuar con ella

La ocultación de información se apoya directamente en la interfaz pública:

- La interfaz pública expone solo lo necesario (public)
- Los detalles internos de implementación (atributos y métodos private) quedan ocultos -> ocultación de información (abstracción)
- El usuario de la clase no puede acceder ni depender del estado interno, solo usar la interfaz

***CLASE

Interfaz pública: Los miembros (atributos/metodos indistinguiblemente) que se ven desde fuera, es decir, los que no están ocultos

3. Brevemente: ¿Por qué hay que ser conscientes y diseñar con cuidado la **interfaz pública** de una clase? ¿Es fácil cambiarla?

Porque es la parte visible y estable del contrato con el resto del sistema. Todo el código que use esa clase dependerá directamente de ella

No es fácil cambiar la interfaz pública ya que:

- Rompe dependencias-> Si otras clases usan tu clase a través de su interfaz pública, cualquier cambio puede hacer que dejen de funcionar.

```
public double getSaldo(),
```

Si cambiamos a:

```
public int getSaldo();
```

O eliminas el método, todo código que lo use fallará (errores de compilación).

Eso es romper dependencias: otros módulos dependen de esa firma.

-Propaga cambios

Ejemplo:

Cambias un método en una clase

10 clases distintas lo usan

Tienes que modificar las 10

-Afecta a la reutilización y compatibilidad

-Puede producir errores sutiles

Ejemplo:

Cambias la lógica interna de un método sin cambiar su firma

El programa sigue compilando

Pero el comportamiento cambia inesperadamente

O:

Cambias una validación

Algunos casos dejan de funcionar correctamente

->Son errores difíciles de detectar porque no rompen el código inmediatamente, pero sí la lógica.

***CLASE

La interfaz pública si se cambia tiene más consecuencias que cualquier cambio en la parte oculta

4. ¿Qué son las invariantes de clase y por qué la ocultación de información nos ayuda?

Las invariantes de clases son condiciones que deben cumplirse siempre para que un objeto esté en un estado válido (sea mutable), antes y después de ejecutar cualquier método público de la clase.

Describen las reglas internas que definen la coherencia del objeto

Ejemplos :

saldo >= 0

edad >= 0

ancho > 0 && alto > 0

La ocultación de información es clave porque:

- Impide modificaciones directas del estado interno

- Centraliza el control del estado

- Facilita la validación
- Reduce errores y estados inconscientes

***CLASE

INVARIANTES DE CLASE-> Condiciones que los objetos de esa clase cumplen o deben cumplir para ser válidos y durante toda la vida del objeto

Ej: Cuenta bancaria debe tener siempre saldo positivo (invariante de clase)

Persona debe tener edad ≥ 0

Rectángulo debe tener ancho y alto > 0

¿Es una invariante de clase decir que una variable tiene que ser un número entero? NO, es el sistema de tipos -> son reglas de negocio o lógica, no del lenguaje.

5. Pon un ejemplo de una clase Punto en Java , con dos coordenadas, x e y , de tipo double , con un método calcularDistanciaAOrigen , y que haga uso de la ocultación de información. ¿Cuál es la interfaz pública de la clase Punto ? ¿Qué significa public y private ?

```
class Punto {                                //Ahora mismo tengo garantizado que una vez se crea no
    private double x;                        //x e y son privados-> ocultación de información
    private double y;

    public Punto (double x, double y){ //Interfaz publica
        this.x=x;
        this.y=y;
    }

    double distanciaAOrigen(){               //No es público, tiene visibilidad por defecto (Solo
        return Math.sqrt(this.x*this.x+this.y*this.y);
    }
}
```

¿Qué significa public y private?

-> PUBLIC

Accesible desde cualquier clase

Forma parte de la interfaz pública

Es lo que “ven” los usuarios de la clase

Ej:

```
public double calcularDistanciaAOrigen()
```

-> PRIVATE

Solo accesible dentro de la propia clase

Oculto los detalles internos

Ej:

```
private double x;
```

6. En Java, ¿A quiénes se pueden aplicar los modificadores `public` o `private` ?

El uso de modificadores de acceso `public` y `private` depende del nivel de estructura.

`Public`: La clase es accesible desde cualquier otra clase en cualquier paquete

`Private`: No se puede aplicar a clases de nivel superior

***CLASE

En java

`LPublic`: clases, atributos y métodos

`LPrivate`: clases internas (clases inferiores) (no los estamos viendo), atributos y métodos

7. En POO, la visibilidad puede ser pública o privada, pero ¿existen más tipos de visibilidad? ¿Qué ocurre en Java? ¿Y en otros lenguajes?

Java es muy específico y ofrece cuatro niveles de visibilidad:

-Public

-Protected

-Default

-Private

Aunque solo usas tres palabras clave, el cuarto nivel es el que queda cuando no escribes nada

Cada lenguaje tiene su propia "personalidad" para manejar la privacidad:

-C# añade niveles mas granulares como internal, protected internal o private protected

-Python no tiene restricciones reales. Todo es público técnicamente, pero se usan convenciones de nombres (`_variable`, `__variable`)

-JavaScript usa el símbolo `#` antes del nombre de la variable para marcarla como privada de forma nativa

***CLASE

En java

-protected, solo se ve desde "subclases" (las veremos en el tema de herencia)

-"packaged-private" o sin modificador, solo se ve desde el paquete

8. Responde: Los miembros de instancia privados de un objeto están ocultos para (a) otras clases o (b) otras instancias, aunque sean de la misma clase. Pon un ejemplo añadiendo un método

calcularDistanciaAPunto(Punto otro) y explica la respuesta.*MIRAR*

***CLASE

¿Para qn esta oculto para las mismas clases o para las instancias del objeto?

```

class Punto (){
    //Ahora mismo tengo garantizado que una vez se crea no se puede modificar
    private double x;
    private double y;

    public Punto (double x, double y){ //Interfaz publica
        this.x=x;
        this.y=y;
    }

    public double distanciaAOrigen(){
        return Math.sqrt(this.x*this.x+this.y*this.y);
    }

    public double distanciaA OtroPunto (Punto otro){
        double dx =this.x-otro.x;
        double dy=this.y-otro.y;
        return Math.sqrt(dx*dx+dy*dy);
    }
}

```

¿Esto compila? -> Si

La a, está oculta para código de otras clases

9. ¿Qué son los métodos "getter" y "setter" en los lenguajes orientados a objetos?

En programación orientada a objetos, los getter y los setter son métodos que sirven para acceder y modificar los atributos (variables) de un objeto de forma controlada

Getter-> obtiene el valor de un atributo

Setter-> cambia el valor de un atributo

***CLASE

"getter" y "setter" permiten dar acceso a atributos privados para obtener su valor o cambiarlo

```

class Punto {
    private double x;
    private double y;

    public Punto (double x, double y){
        this.x=x;
        this.y=y;
    }

    public double distanciaAOrigen(){
        return Math.sqrt(this.x*this.x+this.y*this.y);
    }

    public double distanciaA OtroPunto (Punto otro){
        double dx =this.x-otro.x;
        double dy=this.y-otro.y;
        return Math.sqrt(dx*dx+dy*dy);
    }
    public double get(){
        return this.x;
    }
    public void setX(double x){ //Para modificar el valor recibe el valor al que lo quiero
        this.x=x;
    }
    public String getNombre(){ //Los String se pasan por copia de la referencia
        return this.nombre;
    }
}

```

10. Cuando nos referimos a que la ocultación de información mejora la "seguridad" del programa, ¿nos referimos a que no pueda ser "hackeado"?

No, se refiere a que mejora la seguridad, se habla de seguridad lógica y estructural, no de ciberseguridad. Es decir, proteger el estado interno del objeto de usos incorrectos dentro del propio programa

***CLASE

No, esto no es ciberseguridad, es facilitar una programación con menos 'bugs'

(Un bug es un error o fallo en un programa que hace que no funcione como debería)

11. ¿Qué diferencia hay entre miembro de instancia y miembro de clase? ¿Los miembros de clase también se pueden ocultar?

MIEMBRO DE CLASE (STATIC)

Pertenece a la clase, no a los objetos

Hay una sola copia compartida

```
class Persona {  
    static int contador = 0; // miembro de clase  
}
```

MIEMBRO DE INSTANCIA (NO STATIC)

Pertenece a cada objeto

Cada instancia tiene su propia copia

```
class Persona {  
    private int edad; // miembro de instancia  
}
```

***CLASE

Un miembro de clase no está asociado a ninguna instancia si no que; es compartido por todas las instancias

En métodos, no hay this -> Static

Un miembro de instancia está asociado a cada instancia; no son compartidos -> No static

12. Brevemente: ¿Tiene sentido que los constructores sean privados?

Si, tiene sentido, pero solo en casos concretos, un constructor privado impide que otras clases creen instancias libremente. Eso se usa cuando quieres controlar cómo y cuántos objetos existen

***CLASE

Tiene sentido? A veces

- Un constructor auxiliar oculto, llamado desde otros constructores públicos
- Cuando prefiero usar métodos factoría (inicializador statico) *MIRAR*
- Cuando quiero controlar el nº de instancias

13. ¿Cómo se indican los miembros de clase en Java? Pon un ejemplo, en la clase `Punto` definida anteriormente, para que incluya miembros de clase que permitan saber cuáles son los valores `x` e `y` máximos que se han establecido en todos los puntos que se hayan creado hasta el momento.

Con `static`, lo que significa que pertenecen a la clase y no a cada objeto individual, por lo que son compartidos por todas las instancias

Aplicación a la clase `Punto`:

```
public class Punto {
    private int x;
    private int y;

    // Miembros de clase (static)
    private static int maxX = Integer.MIN_VALUE;
    private static int maxY = Integer.MIN_VALUE;

    // Constructor
    public Punto(int x, int y) {
        this.x = x;
        this.y = y;

        // Actualizar máximos
        if (x > maxX) {
            maxX = x;
        }
        if (y > maxY) {
            maxY = y;
        }
    }

    // Métodos de clase (static) para consultar los máximos
    public static int getMaxX() {
        return maxX;
    }

    public static int getMaxY() {
        return maxY;
    }

    // Métodos normales (no static)
    public int getX() {
        return x;
    }

    public int getY() {
        return y;
    }
}
```

14. Como sería un método factoría dentro de la clase Punto para construir un Punto a partir de dos coordenadas, pero que las redondee al entero más cercano. Escribe sólo el código del método, no toda la clase ¿Has usado static ?

***CLASE

```

class Punto {
    private double x;
    private double y;

    public Punto (double x, double y){    //en el caso de que el constructor es private -> lo
        this.x=x;
        this.y=y;
    }
    //SI QUEREMOS OCULTAR EL CONSTRUCTOR//
    public static Punto nuevoEn (double x, double y){
        return new Punto (x,y);
    }

    //////////////////////////////////////
    ///
    //METODO FACTORIA//////////////////////////////////////
    public static Punto puntoRedondeado(double x, double y){
        return new Punto (Math.round(x), Math.round(y));
    }

    //////////////////////////////////////

    public double distanciaAOrigen(){
        return Math.sqrt(this.x*this.x+this.y*this.y);
    }

    public double distaciaAOtroPunto (Punto otro){
        double dx =this.x-otro.x;
        double dy=this.y-otro.y;
        return Math.sqrt(dx*dx+dy*dy);
    }

    public double get(){
        return this.x;
    }

    public void setX(double x){
        this.x=x;
    }

    public String getNombre(){
        return this.nombre;
    }

}

```

```

class EjercicioEncapsulacion{
    public static void main (String []args){
        Punto p =Punto.puntoRedondeado(4.5,6.7);
        Punto p2=Punto.en(2,3);
    }
}

```

Un método factoría es simplemente un método (static) que se encarga de crear y devolver objetos, en lugar de usar directamente el constructor desde fuera. Sirve para controlar cómo se crean los objetos (por ejemplo, redondeando valores, validando datos, reutilizando instancias, etc.). En Punto hemos usado static correctamente, porque quiero crear un Punto sin necesitar un objeto previo.

DIFERENCIA ENTRE NEW/FACTORIA

Con new creas el objeto directamente sin lógica adicional, mientras que con un método factoría la creación del objeto pasa por un método que puede aplicar lógica (como redondear, validar o modificar valores).

->private en el constructor =

“Nadie desde fuera puede usar new”

Entonces:

“La única forma de crear objetos es la que yo decida (métodos factoría)”

15. Cambia la implementación de Punto . En vez de dos double , emplea un array interno de dos posiciones, intentando no modificar la interfaz pública de la clase. *mirar*

***CLASE

```

class Punto {

    private double []coordenadas = new double [2];

    public Punto (double x, double y){
        this.coordenadas[0]=x;
        this.coordenadas[1]=y;
    }

    public double getX(){                                //Son primitivas, trabajan directamente con la l
        return this.coordenadas[0];
    }

    public double getY(){
        return this.coordenadas[1];
    }

    //----- Arriba primitivas, Abajo derivadas o no primitivas (e:

    public double distanciaAOrigen(){
        return Math.sqrt(this.getX()*this.getX()+this.getY()*this.getY());
    }

    public double distaciaAOtroPunto (Punto otro){
        double dx =this.getX()-otro.getX();
        double dy=this.getY()-otro.getY();
        return Math.sqrt(dx*dx+dy*dy);
    }

    class EjercicioEncapsulacion{
    public static void main (String []args){
        Punto p =new Punto(4,5);
        System.out.println("Tu punto está en: (" + p.getX() + ", " + p.getY() + ")")
        System.out.println ("Distancia: "p.distanciaAOrigen());
    }

}
}

```

16. Si un atributo va a tener un método "getter" y "setter" públicos, ¿no es mejor declararlo público? ¿Cuál es la convención más habitual sobre los atributos, que sean públicos o privados? ¿Tiene esto algo que ver con las "invariantes de clases?"

Si haces el atributo público y además pones getters/setters publicos, en la paractiva no ganas nada: cualquiera puede saltarse los métodos y tocar el estado directamente

La clave es que el getter/setter no solo es un "acceso", es un punto de control

```
private int edad;

public void setEdad(int edad) {
    if (edad < 0) {
        throw new IllegalArgumentException("La edad no puede ser negativa");
    }
    this.edad = edad;
}
```

Si edad fuera pública, este control sería imposible de garantizar

La convención más extendida es: atributos privados, acceso a través de métodos

Eso se suele resumir como: encapsulación, ocultar estado y programar contra interfaces, no contra implementación

Sí, esta totalmente relacionado con las variantes de clases. Estas son reglas que siempre deben cumplirse.

->Si los atributos son privados:

La clase controla los cambios

Puede asegurar que las invariantes se cumplen

->Si son públicos:

Cualquiera puede romper esas reglas

La clase pierde el control

***CLASE

Si los hago publicos:

-Para poder garantizar la variante de clase

-Para poder cambiar la representacion interna

-Convencion es:

atributos siempre privados y emplear métodos de acceso(para poder tener un acceso controlado)
publicos -> encapsulación

17. ¿Qué significa que una clase sea **immutable**? ¿qué es un método modificador? ¿Un método modificador es siempre un "setter"? ¿Tiene ventajas que una clase sea immutable?

1.¿Que significa que una clase sea "immutable"?

Una clase es immutable cuando una vez creado un objeto, su estado no puede cambiar nunca. Es decir, sus atributos no se modifican después del constructor y cualquier "cambio" produce un nuevo objeto, no modifica el existente

```
public final class Dinero {  
    private final BigDecimal importe;    //final= no se puede reasignar-> no hay setters (para m  
    private final String moneda;  
  
    public Dinero(BigDecimal importe, String moneda) {  
        this.importe = importe;  
        this.moneda = moneda;  
    }  
}
```

Una vez creado, en el main: :

```
Dinero d = new Dinero(100, "EUR");  
// no hay forma de cambiar importe ni moneda
```

Entonces, como "cambiarías" algo?

Creando otro objeto:

```
//no es un metodo factoria porque d -> es el objeto existente, sumar(...) -> se ejecuta sobre ese  
public Dinero sumar(BigDecimal cantidad) {  
    return new Dinero(this.importe.add(cantidad), this.moneda);  
}
```

2.¿Qué es un método modificador?

Un método modificador es cualquier método que cambia el estado interno del objeto

```
public void setEdad(int edad) {  
    this.edad = edad;  
}
```

3.¿Un método modificador es siempre un setter?

No, un setter es solo un tipo concreto de método modificador

```
class Contador {  
    private int valor = 0;  
  
    public void incrementar() {  
        this.valor++;  
    }  
}
```

El método incrementar():

No recibe ni asigna un valor directamente como un setter

Pero modifica el atributo valor

Por tanto, es un método modificador

4.¿Tiene ventajas que una clase sea immutable?

Si:

- Invariantes garantizadas
- Mucho más facil de razonar
- Thrad-safe gratis
- Menos efectos colaterales
- Perfectas como value objects

***CLASE

1. IMMUTABLE-> Su estado no cambia

2. Método modificador-> Cualquier método que cambia el estado interno, por ejemplo un setter

3. No, pero un setter si es un metodo modificador. Pero no todos los metodos modificadores son setter
4. Si, las clases inmutables tienen ventajas-> no pongo setters (si los pongo pierdo la inmutabilidad de mi clase)-> no hacer clases mutables comp primera opcion

18. ¿Es recomendable incluir métodos "setter" siempre y como convención?

En general no, solo es recomendable incluirlos cuando tengan sentido. Tener setters siempre y como convención suele ser una mala señal de diseño:

-Rompen la encapsulación ->mal usado, puede debilitar el control que la encapsulación pretende garantizar.

Idea clave

Los setters no rompen la encapsulación; lo que puede romperla es permitir modificar el estado sin control ni validación.

Es decir:

->Setter que respeta la encapsulación:

```
public void setEdad(int edad) {  
    if (edad < 0) {  
        throw new IllegalArgumentException("Edad no válida");  
    }  
    this.edad = edad;  
}
```

->Setter que no respeta la encapsulación:

```
public void setEdad(int edad) {  
    this.edad = edad; //Cualquier valor es válido (incluso negativo), lo que puede romper la  
}
```

-Convierten las clases en un "struct con métodos"

Si una clase tiene muchos atributos públicos o setters sin lógica, se comporta como una simple estructura de datos (como un struct en C), donde solo guardas datos y poco más.

Problema:

La clase pierde "comportamiento real"

Solo actúa como contenedor de datos

-Hacen los objetos frágiles

Como cualquier parte del código puede cambiar el estado del objeto, es difícil garantizar que el objeto siempre esté en un estado válido.

Porque si hay setters públicos, cualquier clase que tenga acceso al objeto puede llamarlos.

Consecuencia:

Puedes romper invariantes sin darte cuenta

El objeto depende demasiado de cómo lo usen desde fuera

Tiene sentido añadir un setter cuando el cambio es simple, no rompe invariantes y representa algo natural del dominio

***CLASE

No, si no ya haria solo clases mutables

19. ¿La clase `String` en Java es mutable o inmutable? ¿Qué ocurre al concatenar dos cadenas? ¿Qué debemos hacer si vamos a hacer una operación que implique concatenar muchas veces para construir paso a paso una cadena muy larga?

1. Con el `String` y el operador `+`

Como el objeto `String` es inmutable, no puedes "estirarlo" ni cambiarle letras.

¿Qué puedes hacer? Cambiar la "flecha" (referencia).

¿Qué ocurre? Al usar `+`, Java crea un objeto nuevo con el resultado y tú reasignas tu variable para que apunte a ese nuevo objeto. El objeto viejo se queda "huérfano" en la memoria.

2. Con el `StringBuilder`

El objeto `StringBuilder` es mutable.

¿Qué puedes hacer? Modificar el contenido del mismo objeto sin crear otros nuevos.

¿Qué ocurre? No necesitas cambiar la referencia de la variable constantemente. La "pizarra" es la misma, solo que ahora tiene más texto escrito.

***CLASE

La clase `String` es inmutable

String elTitulo=libro.getTitulo(); -> Copia de la referencia a titulo

elTitulo="otra cosa" -> estamos cambiando la referencia

elTitulo.charAt(0,"A"); -> Cambia el estado del libro, por lo que este metodo no existe, es immutable , no se puede modificar

Yo no voy a poder encontrar metodos en el String que me permitan cambiar el metodo del String

Los + de Java "hola"+"que tal" son inmutables y crearia un String hola, un String que tal y otro String para la suma de las dos . Osea 3 objetos

si queremos evitar esto-> hacemos un String builder-> ejemplo de un objeto mutable que se utiliza para crear String muy largas

elTitulo.append("mas texto"), añadimos texto y texto al String y luego hacemos .builder para mostrar todo el texto añadido

20. En POO ¿Cómo se comparan objetos de una misma clase? ¿Por su contenido o por su identidad? ¿Qué es el método equals en Java? ¿Qué hace por defecto? ¿Cómo se deben comparar dos cadenas en Java?

1.¿Cómo se comparan objetos de una misma clase en POO? ¿Por su contenido o por su identidad?

Depende de qué quieras comparar:

-Por identidad: Compara si son el mismo objeto en memoria,es decir, si dos referencias apuntan exactamente al mismo objeto (==)

Ejemplo: if (obj1 == obj2)

-Por contenido (estado): Compara si los objetos tiene los mismos valores en sus atributos,aunque sean objetos distintos en memoria (equals())

Ejemplo: if (obj1.equals(obj2))-> Si esta implementada en esa clase, nos devuelve true o false (boolean)

2.¿Qué es el metodo equals en Java?

equals es un método de la clase object, que todas las clases heredan. Su propósito es comparar objetos lógicamente, no por referencia

Tenemos que asegurarnos de que se está haciendo una comparación por contenido

3.¿Cómo se deben comparar dos cadenas en Java?

Las cadenas en Javan se comparan con equals, no con ==

```
if (s1.equals(s2)) { ... }
```

***CLASE

equals: Por defecto hace comparación por identidad (==), excepto en clases concretas donde se implementa una comparación por contenido, p.ej en String

21. ¿Qué son las clases "wrapper" en un lenguaje de programación orientado a objetos? ¿Cómo se hace? ¿Es un proceso automático? ¿Qué ventajas tienen? ¿Todos los lenguajes orientados a objetos tienen tipos primitivos y necesitan wrappers?

Las clases wrapper son clases que envuelven en un tipo de dato primitivo para poder tratarlo como un objeto dentro de un lenguaje orientado a objetos. Permiten usar valores simples (como int, float, char) allí donde se aceptan objetos

En lenguajes que hacen una distinción entre tipos primitivos (almacenados en la pila/stack por eficiencia) y objetos (almacenados en el montón/heap con metadatos), las clases wrapper sirven de puente.

¿Cómo se hace?

Se pueden crear wrappers propios para encapsular otros tipos de datos o crear uno nuevo

```
Integer n = new Integer(10);    // forma antigua  
Integer m = Integer.valueOf(10); // forma recomendada
```

¿Es un proceso automático?

Puede serlo o no, según el lenguaje

En Java existe:

Autoboxing: primitivo → wrapper

Es cuando el compilador de Java detecta que necesitas un objeto, pero tú le das un primitivo. Java, de forma automática, lo "mete en la caja". -> lo que está haciendo Java realmente -> Character letra = Character.valueOf('a');

Unboxing: wrapper → primitivo

Es el proceso inverso. Cuando tienes el objeto (la caja) pero necesitas el valor puro para hacer un cálculo matemático, Java "saca el valor de la caja".

```
Integer caja = 10;
```

```
int suma = caja + 5; // Aquí ocurre el unboxing -> lo q está haciendo Java-> int suma = caja.intValue()  
+ 5;
```

```
Integer a = 5; // autoboxing (int -> Integer)  
int b = a;      // unboxing (Integer -> int)
```

Aquí el programador no hace nada explícito, el compilador lo maneja

¿Que ventajas tiene las clases wrapper?

- Permiten usar primitivos como objetos
- Proveen métodos útiles
- Permiten trabajar con valores nulos
- Compatibilidad con APIs orientadas a objetos

¿Todos los lenguajes orientados a objetos tienen tipos primitivos y necesitan wrappers?

No. Python, Ruby, Smalltalk no tienen primitivos y C++ tiene primitivos pero con un enfoque distinto

Es decir, los wrappers existen porque algunos lenguajes mezclan eficiencia (primitivos) con POO (objetos)

***CLASE -----

Los wrapper ocurren en lenguajes que tienen tipos primitivos (p.ej Java)

Otros lenguajes no tienen tipos primitivos, como Python

No puedo tener un tipo primitivo como objeto????Si:

int <-> Integer

float <-> Float

char <-> Character

Ventajas

- Añadirle comportamiento
- Poder usarlos en contextos donde se necesitan Objetos

List-> Para crear una lista de objetos tienes que crear una lista de Integer (asi tenemos objetos)

Autoboxing/Unboxing

Por detras se estan haciendo las conversiones de forma automatica, a traves del int hacer el integer o viceversa

```
Integer i=7i //Autoboxing -> se produce-> Integer i=new Integer (7);
```

```
int j=i //Unboxing -> int j= i.intValue() -> que me devuelve el contenido del integer
```

22. ¿En POO qué es un **tipo de dato enumerado**? ¿En Java, un tipo de dato enumerado es una clase? ¿Qué ventajas tienen en términos de encapsulación los enumerados en Java?

En POO, un tipo de dato enumerado es un tipo especial que:

- Define un conjunto finito y cerrado de valores posibles
- Cada valor representa una opción válida del dominio
- Evita valores inválidos, mágicos o inconscientes

```
DíaSemana = {LUNES, MARTES, MIÉRCOLES, ...}  
EstadoPedido = {CREADO, ENVIADO, ENTREGADO}
```

En vez de usar int, String o constantes sueltas, el dato enumerado modela explícitamente el concepto

En Java, un enum es una clase.

Ventajas:

- Encapsulan comportamiento, no solo datos
- Control total sobre los valores válidos
- Evitan el uso de constantes públicas dispersas
- Seguridad de tipos
- Encapsulan lógica relacionada al estado

***CLASE

Enumerado es un tipo con un número determinado de valores posibles

En Java un enumerado es una clase, cuyas instancias son finitas, conocidas de antemano, y tienen un nombre cada una (valor del enumerado)

```

public enum TipoIva{
    GENERAL (1.21), REDUCIDO (1.1);
    /*
    public double aplicar (double cnt){
        return switch (this){
            case GENERAL-> return cnt*1.21;
            case REDUCIDO-> return cnt*1.1;
        }
    }
    */
    private double factor;
    private TipoIva(double factor){           //Solo puede ser private, dentro del enum (pub.
        this.factor=factor;                  //Al crear un constructor nos obliga a darle un
    }

    public double aplicar (double cnt){
        return cnt*this.factor;
    }
}

```

23. Crea un tipo enumerado en Java que se llame Mes , con doce posibles instancias y que además proporcione métodos para obtener cuántos días tiene ese mes, el ordinal de ese mes en el año (1-12), empleando atributos privados y constructores del tipo enumerado. Añade además cuatro métodos para devolver si ese mes tiene algunos días de invierno, primavera, verano u otoño, indicando con un booleano el hemisferio (norte o sur, parámetro enHemisferioNorte). Es decir:

esDePrimavera(boolean esHemisferioNorte) ,

esDeVerano(boolean esHemisferioNorte) ,

esDeOtoño(boolean esHemisferioNorte) ,
esDeInvierno(boolean esHemisferioNorte)

```
public enum Mes {
    ENERO(31, 1), FEBRERO(28, 2), MARZO(31, 3), ABRIL(30, 4),
    MAYO(31, 5), JUNIO(30, 6), JULIO(31, 7), AGOSTO(31, 8),
    SEPTIEMBRE(30, 9), OCTUBRE(31, 10), NOVIEMBRE(30, 11), DICIEMBRE(31, 12);

    private final int dias;
    private final int numeroMes;

    // Constructor privado (los enum no pueden ser public)
    private Mes(int dias, int numeroMes) {
        this.dias = dias;
        this.numeroMes = numeroMes;
    }

    // Métodos solicitados
    public int getDias() {
        return dias;
    }

    public int getNumeroMes() {
        return numeroMes;
    }

    // Lógica de estaciones por hemisferio
    public boolean esDePrimavera(boolean enHemisferioNorte) {
        if (enHemisferioNorte) {
            // Marzo, Abril, Mayo, Junio (Norte)
            return numeroMes >= 3 && numeroMes <= 6;
        } else {
            // Septiembre, Octubre, Noviembre, Diciembre (Sur)
            return numeroMes >= 9 && numeroMes <= 12;
        }
    }

    public boolean esDeVerano(boolean enHemisferioNorte) {
        if (enHemisferioNorte) {
            return numeroMes >= 6 && numeroMes <= 9;
        } else {
            return (numeroMes >= 12 || numeroMes <= 3);
        }
    }
}
```

```
    }  
}  
  
public boolean esDeOtoño(boolean enHemisferioNorte) {  
    return esDePrimavera(!enHemisferioNorte); // El otoño es la primavera del otro lado  
}  
  
public boolean esDeInvierno(boolean enHemisferioNorte) {  
    return esDeVerano(!enHemisferioNorte); // El invierno es el verano del otro lado  
}  
}
```